Computación Concurrente, Paralela y Distribuida, Práctica 3

Paralelismo implementado con Pooling

Versión: 1 de diciembre de 2025

Contexto

Con esta práctica dejaremos atrás la concurrencia y comenzaremos a ejecutar **programas auténticamente paralelos**, aunque mantendremos el uso de **hilos** a fin de compararlos con los **procesos**. Por norma general, en los programas paralelos que veremos a partir de ahora, en esta práctica y en las siguientes, tendremos una cantidad de **datos** que deberán ser **divididos** para luego ser **procesados** entre un número configurable de procesos o de hilos. Con esta división buscaremos la **esencia del paralelismo**, **mejorar** los **tiempos** de ejecución de una forma proporcional a los recursos usados.

Para poder mejorar estos tiempos aplicaremos **numerosas técnicas** distintas, cada una con sus ventajas y sus inconvenientes, empezando en esta práctica con la técnica de *Pooling*, y siguiendo con otras en las futuras prácticas. Lo que no cambiará respecto a la anterior práctica será la **estrategia de experimentación**, que seguirá siendo la **ejecución** con diversas **configuraciones**, aunque ahora se pondrá especial énfasis en el grado de paralelismo o su tipo. De igual forma, de ahora en adelante con los programas paralelos será crucial añadir una fase de **validación** de resultados, así como una constante **comparación de rendimiento** con un escenario base representado por una ejecución secuencial.

Objetivos

En esta práctica usaremos un código donde se usa la técnica de *Pooling* para paralelizar una carga de trabajo, con los siguientes objetivos:

- Ver la posibilidad de **procesar una carga** de trabajo que requiere computación real (no se usan *sleeps*) **de forma paralela** tanto con hilos como con procesos. Pondremos especial énfasis en la **validación** como herramienta base para ver si el paralelismo se implementó de forma correcta. Además, analizaremos la *justificación del uso y reuso de un conjunto de procesos*, en lugar de su creación y destrucción bajo demanda.
- Explorar distintos escenarios que expondrán varios *límites de paralelismo*, tanto aquellos que no controlamos (e.g., **software**) como algunos que podemos controlar hasta cierto punto (e.g., **datos** y **hardware**).
- Ya más en el aspecto técnico, se introducirá el uso de una **máquina remota** que permitirá aumentar el límite de paralelismo esperable de los recursos del alumnado, con la contrapartida de que dicha máquina representará un **recurso compartido** entre dicho alumnado y requerirá de unas nociones básicas en su uso, así como entender la interacción con un **sistema de colas**.

Índice

1 Conteo usando *Pooling* **3**
- 1.1 Explicación del código . . . 3
- 1.2 Validación . . . 4
- 1.3 Justificación del uso de Pools . . . 5

2 Límites de paralelismo **7**
- 2.1 Límite software (GIL) . . . 7
- 2.2 Límite por tamaño de datos . . . 9
- 2.3 Límite de recursos hardware . . . 10

3 Recurso compartido, la Máquina Remota **12**

Índice de Tareas

Tarea 1: Análisis del código y sus secciones 4
Tarea 2: Ejecución del código, prueba de validación 5
Tarea 3: Ejecución del código, justificación del uso de Pools 5
Tarea 4: Límite impuesto por el GIL 7
Tarea 5: Límite impuesto por el tamaño de la carga 9
Tarea 6: Límite impuesto por los recursos hardware 10
Tarea 7: Ejecución en la Máquina Remota 12

Índice de figuras

1 Salida de ejecución y gráficos generados del experimento sobre el GIL 8
2 Salida de ejecución y gráficos generados del experimento sobre el tamaño de los datos 9
3 Salida de ejecución y gráficos generados del experimento sobre el límite hardware 11
4 Salida de ejecución del experimento sobre el límite hardware en la MR 12
5 Gráficos generados del experimento sobre el límite hardware en la MR 13

Antes de empezar: En esta práctica se introducirá el uso de una máquina remota, para la cual se aconseja leer su guía de uso **antes de usarla**. El uso de esta máquina se hará mediante scripts que realizan conexiones por SSH, para lo cual es necesario que el contenedor tenga una llave SSH en el directorio '/home/alumno/.ssh'. Esto lo podremos hacer mediante el script 'deploy-keys.sh' que encontrarás en Moodle..

1. Conteo usando *Pooling*

Siguiendo con la línea de ejemplos de prácticas anteriores, en el programa que encontrarás en el *código1* se ha implementado un problema muy sencillo, para cada número de una lista se lleva una cuenta acarreada dentro de un rango definido. No obstante, dichos números de la lista son independientes entre sí a la hora de llevar la cuenta, por lo que representarán en última instancia tareas divisibles y paralelizables. Esto último será precisamente aprovechado para procesar los datos mediante el uso de la técnica de *Pooling*.

Esta técnica se refiere de forma genérica al uso de conjuntos (pools). En nuestro ámbito podemos referirnos a este conjunto para representar tanto un conjunto de trabajos a realizar (**pool de tareas**), como un conjunto de ejecutores (**pool de ejecutores**), ya sean hilos o procesos, que llevarán a cabo dichas tareas. En los códigos con los que trabajaremos tendremos que tener en cuenta ambos tipos. Aunque la definición en el código se hace de un **pool de ejecutores**, configurado con un tamaño según el grado de paralelismo que queramos aplicar (variable **N_WORKERS_LIST**), también tendremos que considerar el número de tareas que lanzamos para ser ejecutadas. El número y la complejidad de estas tareas, así como la proporción de tareas y ejecutores (ratio del tamaño de los pools), serán factores clave de los que dependerá el rendimiento.

Finalmente, hay que tener en cuenta que en el aspecto técnico, la técnica de *Pooling* permite el aprovechamiento de recursos al re-usar procesos o hilos, en vez de crearlos y destruirlos constantemente. Esta diferencia será la base de la existencia de esta técnica.

1.1. Explicación del código

Para formar el **pool de ejecutores**, en este caso usaremos los 2 tipos existentes, tanto los basados en hilos como los basados en procesos, como se puede ver en las siguientes líneas del *código1*.

Creación de pools basados en procesos e hilos (codigo1.py)

```
108     pools = []
109     if USE_PROCESSES:
110         pools.append((ProcessPoolExecutor, "PRO"))
111     if USE_THREADS:
112         pools.append((ThreadPoolExecutor, "THR"))
```

Uso de pools (codigo1.py)

```
130             for PoolType, PoolLabel in pools:
```

En lo que respecta a las tareas que queremos ejecutar de forma paralela, de momento no las agruparemos y las ejecutaremos de forma individual. Dichas tareas las lanzaremos en las siguientes líneas, usando la variable 'executor' que representa el pool, y su función 'submit()'. Estas tareas estarán a su vez representadas por variables individuales, denominadas habitualmente 'futuros o promesas', que nos permitirán interactuar con las tareas si es necesario:

Reparto de las tareas en el Pool (codigo1.py)

```
138                     with PoolType(max_workers=workers_num) as executor:
139                         futures = list()
140                         for item in tasks_list:
141                             futures.append(executor.submit(evaluate_item, item, count))
```

No obstante, hay que tener en cuenta que estas líneas meramente definen un Pool y lanzan tareas sobre él, pero obviamente dichas tareas (o futuros), que serán ejecutadas por procesos o hilos, necesitarán un tiempo para completarse (de ahí su nombre). Dado que la creación del Pool y el lanzado de tareas son funciones que realiza el hilo maestro, tendremos que hacer una gestión de la tareas hijas. Por suerte esta tarea será sencilla, básicamente esperar a sus resultados, recogerlos y guardarlos de forma acorde, lo que se hace en las siguientes líneas:

Recogida de resultados (codigo1.py)

```
142                         result_items = list()
143                         for f in futures:
144                             result_items.append(f.result())
```

Finalmente, en lo que respecta a la ejecución secuencial, la podrás encontrar en las siguientes líneas. Este código representará el *escenario base*, del que usaremos su resultado y tiempo de ejecución para comparar con los resultados de las ejecuciones paralelas, tanto a fin de validación como de medición del rendimiento.

Versión secuencial (codigo1.py)

```
120             for item in tasks_list:
121                 result_items.append(evaluate_item(item, count))
```

Tarea 1 – Análisis del código y sus secciones

Analiza el resto del código, intenta identificar las secciones de códigos usadas para:

- **La resolución del problema (secuencial y paralelo)**
- **Gestión del paralelismo**
- **Gestión experimental o de configuración**
- **Impresión de resultados (tablas o gráficos)**

***Cláusula 'with'*:** La cláusula 'with' es usada en Python para representar la apertura de una variable o estructura de control que requiere una gestión específica, más específicamente, de 'limpieza' al acabar. Es muy común verla siendo usada con la apertura de ficheros para lectura y escritura, dejando en ese caso a la cláusula 'with' la gestión del cierre del fichero una vez finalizada la operación de lectura o escritura. En el *código1*, más específicamente, podemos verla siendo usada para la gestión del Pool, dado que este requiere ser cerrado una vez que se termina de trabajar con el (parada y destrucción de procesos o hilos), así como para la gestión de la barra de progreso de la librería tqdm.

1.2. Validación

En lo que respecta a la validación de resultados, al aplicar paralelización en cualquier programa será necesario que comprobemos después que los resultados de procesar un input dado siguen siendo los mismo (o aceptables),

que los de su versión secuencial. Si en algún momento no se pasara esta prueba de validación, cualquier resultado que obtuviéramos de la paralelización, por bueno que fuese, quedaría **invalidado**. A su vez, usaremos el *escenario base* para referirnos a una ejecución en la que confiamos y cuyos resultados son correctos. Nos servirá tanto para llevar a cabo la *validación de resultados*, como para hacer una *comparación de rendimiento* que nos permita saber si el uso de paralelismo está o no justificado.

Tarea 2 – Ejecución del código, prueba de validación

Descomenta y ejecuta el escenario experimental denominado "VALIDITY TEST". Comprueba que el resultado de la ejecución es el mismo con la solución secuencial (columna SEQ) y con la paralela (columnas 2x y 4x para hilos y procesos).

Por el momento puedes ignorar las columnas 'PRO_per_task'.

```
alumno@ParalelismoMV:~/Compartida/P3-Pooling$ python3 codigo1.py
VALIDITY TEST
@@@@@@@@@@@@@@@@@@

OUTPUT          -> True
NUM_LIST_LEN    -> 5
N_WORKERS_LIST  -> [2, 4]
COUNT_INI       -> 1e3
COUNT_MAX       -> 1e4
RESULTS:
Resulting items of SEQ are -> [49995, 54984, 59985, 64980, 69975]
Resulting items of PRO_2 are -> [49995, 54984, 59985, 64980, 69975]
Resulting items of PRO_4 are -> [49995, 54984, 59985, 64980, 69975]
Resulting items of THR_2 are -> [49995, 54984, 59985, 64980, 69975]
Resulting items of THR_4 are -> [49995, 54984, 59985, 64980, 69975]

RESULTS:
Resulting items of PRO_per_task_2 are -> [49995, 54984, 59985, 64980, 69975]
Resulting items of PRO_per_task_4 are -> [49995, 54984, 59985, 64980, 69975]

TIMES (ms):
     SEQ  PRO_2  PRO_4  THR_2  THR_4  PRO_per_task_2  PRO_per_task_4
1e3    0     11     14      1      1              25              21
1e4    9     15     18      9      9              30              23
```

Si la prueba de validación es exitosa y los resultados coinciden, tenemos un código que es 'potencialmente' correcto y paralelo. Si las librerías de paralelismo no contienen errores, y si el diseño de nuestro programa tampoco los contiene a la hora de gestionar el paralelismo (principalmente comunicaciones y reparto de trabajo), podemos asumir que el programa funcionará con cualquier grado de paralelismo (e.g., 2x, 4x, 7x, 200x).

Esta asunción nos permitiría usar la técnica ampliamente usada de *'testear con 8x, ejecutar con 800x'*, es decir, programar y probar a 'pequeña escala', para luego ejecutar un problema con una dimensión realista a 'gran escala'. Si el rendimiento de nuestro programa se mantiene y progresa acorde al número de recursos, diremos que nuestro programa es además escalable, una característica muy deseable para los programas paralelos y que estudiaremos de forma simple en esta práctica, y de forma más extendida en la siguiente.

1.3. Justificación del uso de Pools

Antes de continuar usando los Pools de procesos, podríamos analizar si dicho uso está realmente justificado en lugar de simplemente crear y destruir procesos bajo demanda o necesidad. La realidad es que la reutilización de procesos está más que justificada en el momento en el que se espera ejecutar un número de tareas uno o varios órdenes de magnitud superior (e.g., 10.000) al de procesos arrancados en un Pool (e.g., 32). Esto es debido a que la creación y destrucción de un proceso tiene un coste asociado, y mientras que con el uso de un Pool estos costes solo se darán una vez, si se crean y destruyen para cada tarea se estarán pagando continuamente. Aunque para este apartado dejaremos de lado los hilos, el argumento aplicaría de igual forma que con los procesos.

Tarea 3 – Ejecución del código, justificación del uso de Pools

Descomenta ahora el experimento "EFFECT OF PROCESS CREATION/DESTRUCTION...", en el cual se explora esta diferencia en la gestión de procesos e hilos. En este escenario vamos a ejecutar 2.000 tareas con 2 métodos y comparar el tiempo que llevan.

Por un lado las ejecutaremos usando un Pool de procesos. Por otro lado dividiremos las tareas en lotes, y luego arrancaremos un proceso para cada tarea del lote, que será luego destruido y no reutilizado para el siguiente lote. El motivo de procesar las tareas en lotes, en lugar de arrancar tantos procesos como tareas y ejecutarlos todos simultáneamente, es por la potencial limitación del sistema operativo a la hora de arrancar excesivos procesos en un corto espacio de tiempo.

- **Ejecuta el experimento y analiza los resultados de la paralelización con procesos (columnas PRO_x), con la paralelizacion creando/destruyendo procesos (columnas PRO_per_task_X).**
- **¿Qué método consume más?**
- **Dentro del método que crea y destruye procesos, ¿hay diferencias entre agrupar tareas y usar 4 procesos u 8?**

2. Límites de paralelismo

Por desgracia, aún cuando nuestro programa tenga por diseño un paralelismo potencial ilimitado, la realidad es que en ningún caso llegaremos a él y alcanzaremos en todas las ejecuciones unos *límites de paralelismo*. Estos límites podrán a veces estar impuestos por los datos y su tamaño, otras veces por el software, pero muy a menudo en nuestro caso se dará por los recursos hardware disponibles. En las siguientes tareas verás ejemplos de estas limitaciones.

Gráficos y Visualización: Aunque los resultados de las ejecuciones se mostrarán por pantalla en formato de tablas (como en los códigos previamente vistos), también es posible usar la librería de gestión de datos *Pandas* y la de visualización y generación de gráficos *Matplotlib*, para poder ver los mismos resultados de manera más rápida. En algunos códigos con los que trabajaremos, al finalizar la ejecución de los mismos, se mostrarán los resultados por pantalla en forma de tabla y se generarán gráficos de dichos datos.

2.1. Límite software (GIL)

Tarea 4 – Límite impuesto por el GIL

Lo cierto es que **Python NO permite la ejecución simultánea de más de 1 hilo**, algo que restringe mediante el uso de un 'Lock' global de acceso al intérprete. Esto implica que realmente no se puede lograr una ganancia significativa con programas paralelos basados en hilos con Python 'clásico', aunque no todo está perdido realmente.

Este lock, llamado "GIL"(Global Interpreter Lock), solo afecta a las partes del código que lleven a cabo operaciones en el intérprete, algo que excluye a cualquier operación bloqueante (e.g., sleeps, acceso a colas, lectura de datos de red o disco, peticiones a servidores remotos...). Esto es lo que nos ha permitido usar los hilos para obtener ganancias en los programas concurrentes vistos en prácticas anteriores. **Por otro lado, los procesos NO se verán afectados por este Lock al contar cada proceso con su propio intérprete**.

Descomenta el experimento "EFFECT OF THE GIL", en el cual veremos un límite impuesto por el propio entorno de Python, es decir un *límite sofware*. **Ejecuta este experimento para poner de manifiesto la acción del GIL al comparar los tiempos secuenciales con los paralelos usando procesos e hilos.**

```
EFFECT OF GIL
@@@@@@@@@@@@@@@@@

OUTPUT          -> False
NUM_LIST_LEN    -> 32
N_WORKERS_LIST  -> [2, 4, 6, 8]
COUNT_INI       -> 1e3
COUNT_MAX       -> 1e6

TIMES (ms):
      SEQ  PRO_2  PRO_4  PRO_6  PRO_8  THR_2  THR_4  THR_6  THR_8
1e3     5     15     20     27     31      6      7      7      8
1e4    52     39     32     33     38     55     54     57     57
1e5   540    320    217    235    107    536    523    526    516
1e6  5372   2712   1454   1005    737   5049   5255   5565   6026
```

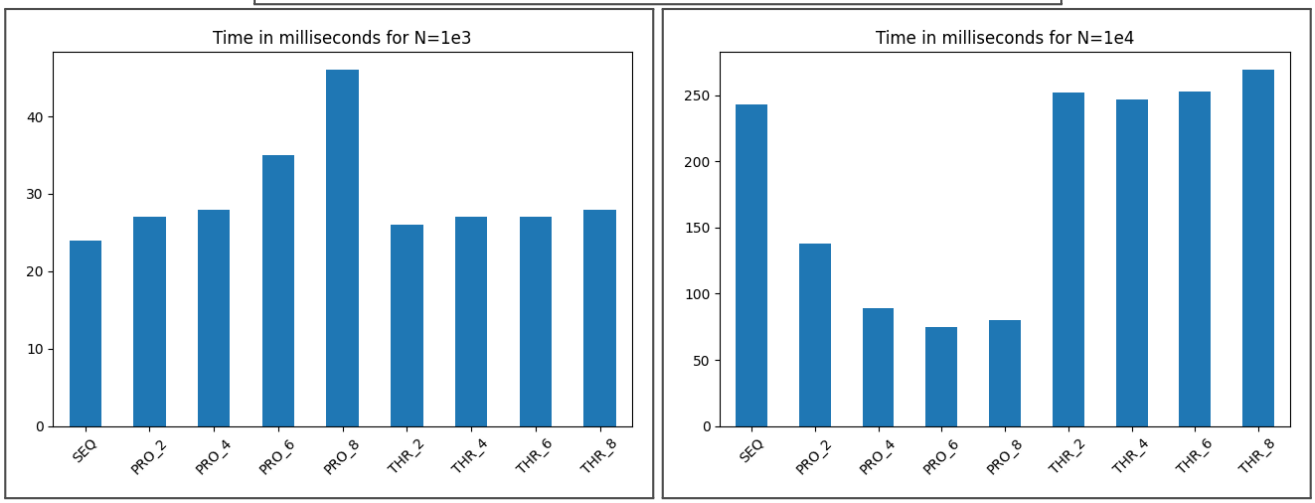

Figura 1: Salida de ejecución y gráficos generados del experimento sobre el GIL

2.2. Límite por tamaño de datos

Tarea 5 – Límite impuesto por el tamaño de la carga

Descomenta ahora el experimento "NUM TASKS SMALLER...", en el cual se explora un *límite impuesto por los propios datos* y su tamaño. En este escenario vamos a simular que tenemos un Pool de ejecutores más grande (de 2x a 6x, ver variable **N_WORKERS_LIST**), que el propio Pool de tareas (4 tareas, ver variable **NUM_LIST_LEN**). Por comodidad descartaremos ya el uso de hilos al haber visto su no aplicabilidad para paralelismo real.

- **Ejecuta el experimento y analiza los resultados de la paralelización con procesos.**
- **¿En qué punto (# de procesos) se deja de ver una ganancia? ¿Por qué es esto?**
- **¿Por qué los tiempos para 2 y 3 procesos son tan similares a pesar de tener una configuración más recursos que la otra?** [Pregunta de mayor dificultad]

```
NUM TASKS (4) SMALLER THAN POOL (6)
@@@@@@@@@@@@@@@@@@

OUTPUT          -> False
NUM_LIST_LEN    -> 4
N_WORKERS_LIST  -> [2, 3, 4, 5, 6]
COUNT_INI       -> 1e3
COUNT_MAX       -> 1e7

TIMES (ms):
      SEQ  PRO_2  PRO_3  PRO_4  PRO_5  PRO_6
1e3     0     12     16     19     23     26
1e4     8     18     18     24     25     25
1e5    75     53     47     38     44     44
1e6   647    386    337    213    200    194
1e7  6782   3294   3290   1849   1684   1655
```

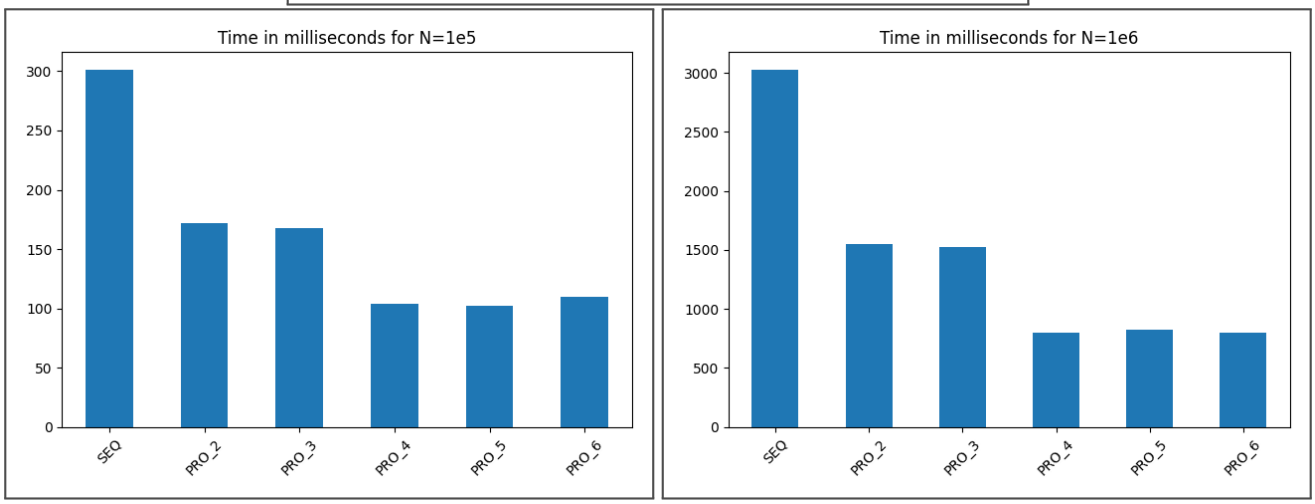

Figura 2: Salida de ejecución y gráficos generados del experimento sobre el tamaño de los datos

2.3. Límite de recursos hardware

Tarea 6 – Límite impuesto por los recursos hardware

En lo que respecta a un *límite hardware*, este será a partir de ahora el más común, y lo veremos alcanzar tanto en nuestra máquina local, como en una máquina remota con (posiblemente) más recursos hardware.

Descomenta el experimento "TRUE PARALLELISM", en el cual aumentaremos de forma significativa tanto la carga de trabajo como el número de trabajadores que usaremos. Para modular esta carga de trabajo no aumentaremos el número de tareas de forma significativa, pero si la cuenta que tendrán que hacer los hilos o procesos (ver variables **COUNT_INI** y **COUNT_MAX**). En lo que respecta al número de trabajadores, lo aumentaremos de forma intencionada hasta unos valores que muy probablemente excederán los recursos del equipo, buscando así alcanzar el límite hardware. **Ejecuta el experimento (puede tardar un poco) y analiza la salida.**

- **¿Por qué, con cargas bajas como N=1e3 o N=1e4, los tiempos de procesos son superiores al secuencial?**
- Visualiza los gráficos en la carpeta 'true_paralellism'. **¿Cual es la evolución del rendimiento con procesos? ¿Cual es tu *límite de paralelismo*?**

```
TRUE PARALLELISM
@@@@@@@@@@@@@@@@@

OUTPUT          -> False
NUM_LIST_LEN    -> 64
N_WORKERS_LIST  -> [2, 4, 8, 16, 24, 32, 64]
COUNT_INI       -> 1e2
COUNT_MAX       -> 1e6

TIMES (ms):
       SEQ  PRO_2  PRO_4  PRO_8  PRO_16  PRO_24  PRO_32  PRO_64
1e2      1     19     22     34      55      81     100     196
1e3     11     20     24     38      56      79     100     219
1e4    107     68     47     49      67     107     115     199
1e5   1039    534    313    179     193     214     237     326
1e6  10388   5263   2781   1492    1430    1447    1474    1558
```

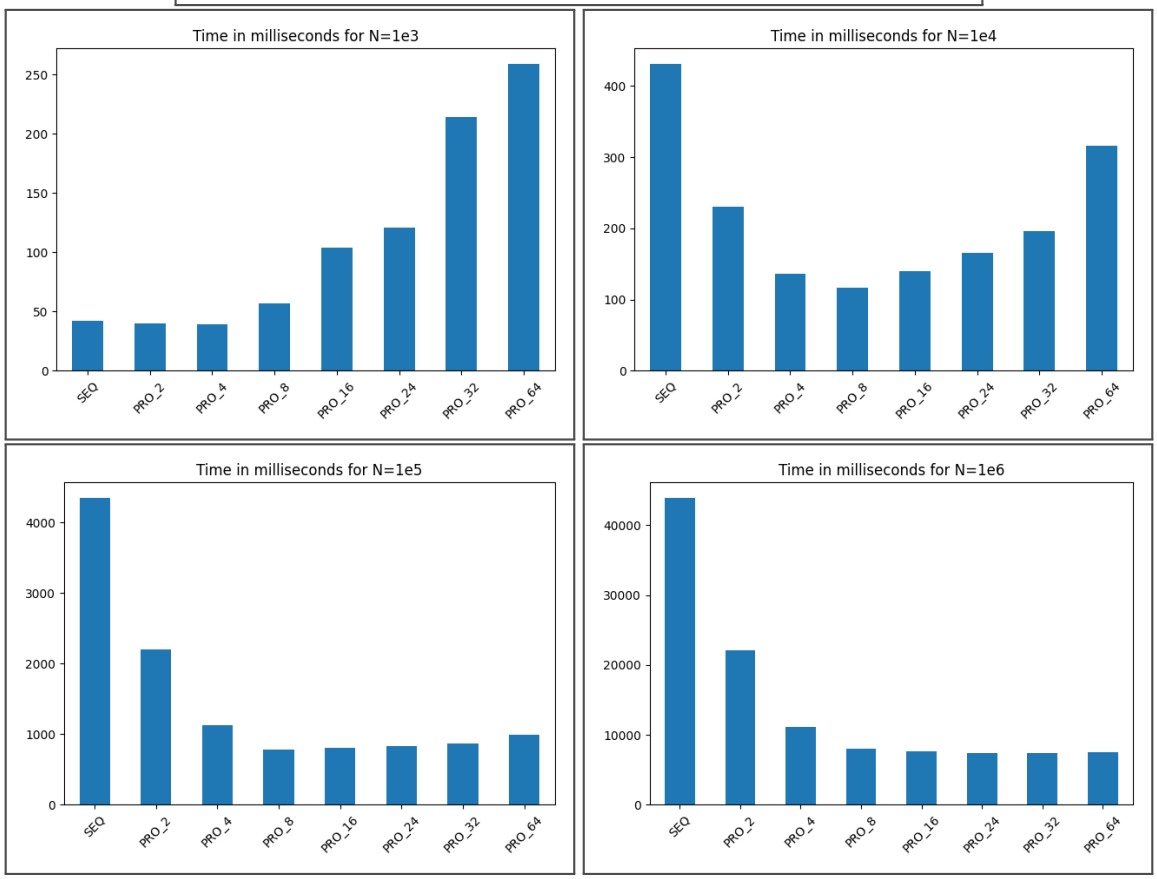

Figura 3: Salida de ejecución y gráficos generados del experimento sobre el límite hardware

3. Recurso compartido, la Máquina Remota

Tarea 7 – Ejecución en la Máquina Remota

Ejecuta ahora el mismo experimento pero en la máquina remota, no obstante, si no lo has hecho, **LEE ANTES LA GUÍA DE USO** de la Máquina Remota (MR).

- Ejecuta el codigo1 en la MR con el script '*execute_codigo1_remotely.sh*' y consulta su salida
- Tráete los gráficos a local con el script '*retrieve_plots.sh*' y visualízalos.
- **¿Cuál es ahora la evolución del rendimiento con procesos?**
- **¿Cuál es el nuevo *límite de paralelismo*?**

```
TRUE PARALLELISM
@@@@@@@@@@@@@@@@

OUTPUT          -> False
NUM_LIST_LEN    -> 256
N_WORKERS_LIST  -> [2, 4, 8, 16, 24, 32, 64, 48, 64, 128, 256]
COUNT_INI       -> 1e2
COUNT_MAX       -> 1e7


TIMES (ms):
        SEQ    PRO_2    PRO_4   PRO_8   PRO_16   PRO_24   PRO_32   PRO_64   PRO_128   PRO_256
1e2       4       42       46      59       82      106      140      235       461       953
1e3      47       48       49      58       85      112      139      246       453       911
1e4     461      258      149     100      114      126      156      251       460       919
1e5    4245     2315     1165     652      417      383      423      480       676      1102
1e6   41978    22083    11357    6147     3187     2924     2747     2863      3068      3434
1e7  420015   220591   113419   60563    31313    28473    27020    26467     26656     27293
```

Figura 4: Salida de ejecución del experimento sobre el límite hardware en la MR

Límite de paralelismo:

Como has podido observar en los diferentes gráficos y experimentos, **siempre** habrá un *límite de paralelismo*. Aunque nuestro programa se comporte bien y escale y paralelice de forma ideal, la realidad es que los recursos hardware serán siempre finitos, o que en algún momento algún tipo de *overhead* (e.g., comunicaciones, creación de procesos) provocará que aumentar el paralelismo no mejore de forma relevante los tiempos respecto a unos resultados anteriores con menor grado de paralelización.

En los experimentos expuestos es común ver cómo se alcanza un límite al acercarse al número de cores, o como mucho, al de hilos hardware, disponibles en la máquina. Este límite siempre suele estar algo por encima de dicho número, algo que puede estar motivado por el efecto beneficioso de un ligero *oversubscription* de recursos (o procesos) para compensar cualquier pérdida de rendimiento asociada a tareas bloqueantes subyacentes al programa.

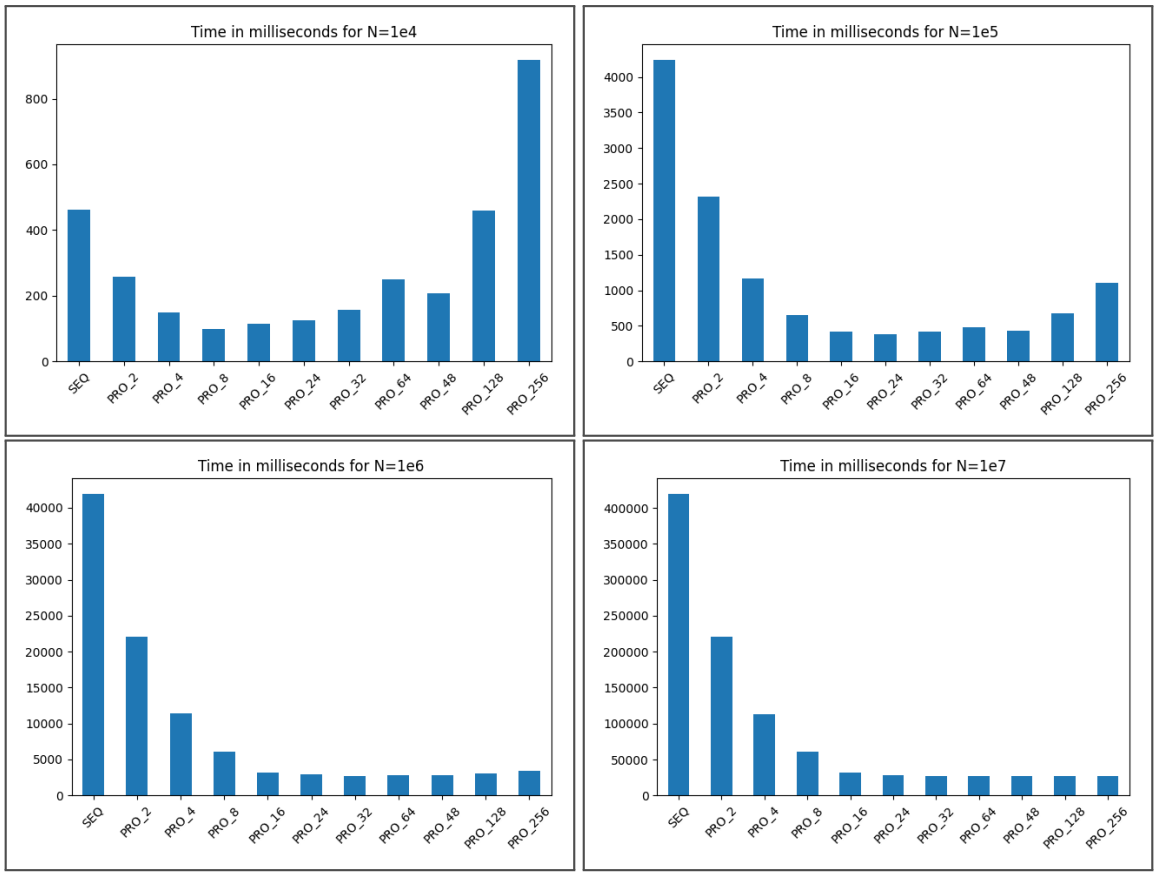

Figura 5: Gráficos generados del experimento sobre el límite hardware en la MR

Copyright & Disclaimer

```
# Copyright © 2025 by Jonatan Enes (jonatan.enes@udc.es)
# Computer Engineering department, Universidade da Coruña, Spain.
#
# This file is part of several courses on parallel processing from Universidade da Coruña,
# and it can only be used and/or modified by the author, the students or any
# explicitly authorized person by the author, inside the context of the subject.
# Redistribution to a third-party without previous authorization is strictly forbidden.
# No commercial use is allowed.
#
# This file has only academic purposes and should not be used for any real-world
# scenario, the author holds no accountability for its use or misuse.
```